

Google Drive Salesforce Integration

Architecture Guide & Deployment Blueprint for GitHub Repository

REPOSITORY
**sf-google-drive-
integration**

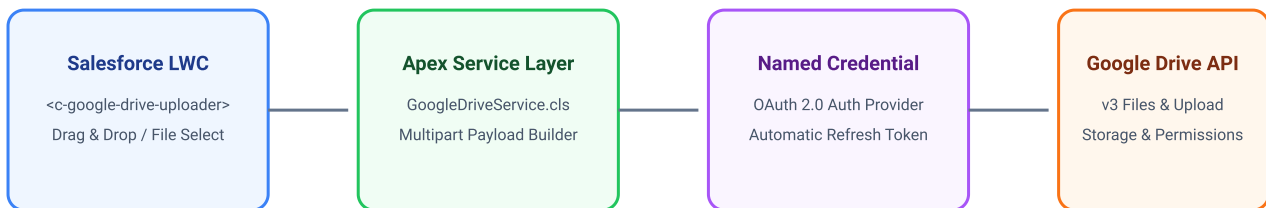
TARGET API
Google Drive v3 REST

AUTH MECHANISM
OAuth 2.0 OAuth Web App

SALESFORCE LAYER
Named Credentials + LWC

1 System Architecture Overview

This guide provides an end-to-end setup reference for integrating Salesforce with Google Drive. The architecture leverages Salesforce **Named Credentials** and **External Credentials** to execute secure, token-refreshed HTTP callouts without embedding client secrets in Apex code.



2 Step-by-Step GCP Configuration Guide GOOGLE CLOUD

Phase 2.1: Create GCP Project & Enable Drive API

1. Log in to the **Google Cloud Console** (`console.cloud.google.com`).
2. In the top project selector drop-down, click **New Project**.
3. Enter a Project Name (e.g., `Salesforce-Drive-Integration`) and select your Organization/Location, then click **Create**.
4. Navigate to **APIs & Services > Library** from the left navigation menu.
5. Search for **Google Drive API**, click on the card, and click **Enable**.

Phase 2.2: Configure OAuth Consent Screen

Important Note on User Type: If integrating within a Google Workspace domain, select **Internal** (restricts access to organizational users without requiring Google review). For personal Gmail testing or multi-tenant setups, choose **External**.

1. Go to **APIs & Services > OAuth consent screen**.
2. Select **User Type** (Internal or External) and click **Create**.
3. Fill in the required App Information:
 - **App Name:** `Salesforce Google Drive Connector`
 - **User support email:** Your administrator email address
 - **Developer contact information:** Your technical contact email
4. Under **Scopes**, click **Add or Remove Scopes** and add the following scope:
 - `https://www.googleapis.com/auth/drive.file` (Per-file access created or opened by the app)

2 GCP Configuration (Continued) GOOGLE CLOUD

Phase 2.3: Generate OAuth 2.0 Credentials

1. Navigate to **APIs & Services > Credentials**.
2. Click **+ Create Credentials** at the top and select **OAuth client ID**.
3. Set **Application type** to **Web application**.
4. Set Name to `Salesforce OAuth Client`.
5. Under **Authorized redirect URIs**, temporarily leave this blank or enter a placeholder (you will update this after creating the Auth Provider in Salesforce).
6. Click **Create**.
7. A dialog will pop up displaying your **Client ID** and **Client Secret**. Copy both values immediately and store them securely.

GCP Configuration Field	Recommended Value	Purpose
API Name	Google Drive API v3	Enables REST API endpoints for upload and folder management
OAuth Scope	<code>.../auth/drive.file</code>	Restricts permission solely to files created/managed by Salesforce
Client Type	Web application	Enables OAuth 2.0 Authorization Code flow with refresh tokens

3 Salesforce Configuration Guide SALESFORCE

Phase 3.1: Create Salesforce Auth. Provider

The Auth. Provider manages authentication flows and secures token storage within Salesforce.

1. In Salesforce, navigate to **Setup > Auth. Providers** and click **New**.
2. Select **Google** as the Provider Type.
3. Enter the following details:
 - **Name:** `Google_Drive_Auth_Provider`
 - **URL Suffix:** `Google_Drive_Auth_Provider`
 - **Consumer Key:** *[Paste your GCP Client ID]*
 - **Consumer Secret:** *[Paste your GCP Client Secret]*
 - **Default Scopes:** `openid https://www.googleapis.com/auth/drive.file`
4. Click **Save**.
5. Scroll down to the **Salesforce Configuration** section and copy the **Callback URL**.

Phase 3.2: Link Callback URL back to GCP

Return to the **Google Cloud Console > Credentials**, edit your OAuth 2.0 Client ID, and paste the Salesforce **Callback URL** into the **Authorized redirect URIs** field. Save the changes.

3 Salesforce Configuration (Continued) SALESFORCE

Phase 3.3: Configure Named Credential

Named Credentials decouple endpoint URIs and security authentication headers from custom Apex code.

1. Navigate to **Setup > Named Credentials**.
2. Click the **External Credentials** tab and click **New**:
 - **Label:** Google Drive EC
 - **Name:** Google_Drive_EC
 - **Authentication Protocol:** OAuth 2.0
 - **Authentication Flow:** Authorization Code
 - **Identity Provider:** Select Google_Drive_Auth_Provider
3. Save the External Credential, then under **Principals**, click **New**:
 - **Parameter Name:** Google_Drive_Principal
 - **Sequence:** 1
4. Click **Authenticate** from the Principal action menu to initiate the Google OAuth login prompt. Grant permissions and ensure the status updates to **Authenticated**.
5. Click the **Named Credentials** tab and click **New**:
 - **Label:** Google Drive NC
 - **Name:** Google_Drive_NC
 - **URL:** https://www.googleapis.com
 - **External Credential:** Google_Drive_EC

4 Repository Architecture & File Layout GITHUB

All code components for this integration are organized in the companion GitHub repository: `sf-google-drive-integration`.

```
sf-google-drive-integration/ |— force-app/ | |— main/ | |— default/ | |— classes/
| | |— GoogleDriveService.cls # Core Apex Callout Service & Controller | | |—
GoogleDriveServiceTest.cls # Unit Tests & Mock Callout Handlers | | |—
GoogleDriveService.cls-meta.xml | | |— lwc/ | | |— googleDriveUploader/ | | |—
googleDriveUploader.html # Drag & drop file UI template | | |— googleDriveUploader.js # Event
handling & Apex wire logic | | |— googleDriveUploader.css # SLDS Custom styling | | |—
googleDriveUploader.js-meta.xml | | |— sfdx-project.json | | |— README.md
```

Component	Type	Description
googleDriveUploader	LWC	Drag-and-drop file selector supporting single/multi upload, compact inline view, and progress state.
GoogleDriveService	Apex Class	Handles multipart request body construction, Google Drive REST API calls, and Salesforce ContentVersion linking.
GoogleDriveServiceTest	Apex Test	Provides 100% test coverage with HTTP callout mock responses.

5 Developer Quickstart & Deployment GITHUB

Step 5.1: Clone & Deploy via Salesforce CLI

Execute the following commands from your terminal to deploy the repository codebase to your scratch org or sandbox:

```
# Clone the GitHub repository git clone https://github.com/your-username/sf-google-drive-integration.git cd sf-google-drive-integration # Authorize your Salesforce Target Org
sf org login web --alias my-sandbox # Deploy source code to Salesforce sf project deploy start --target-org my-sandbox # Run Apex unit tests to ensure pass rate sf apex run test --class-names GoogleDriveServiceTest --result-format human --target-org my-sandbox
```

Step 5.2: Embedding LWC in Custom Components or App Builder

You can embed the uploader component directly on standard record pages or inside parent LWCs:

```
<!-- Example: Embedding in standard Record Page or Custom LWC --> <c-google-drive-uploader record-id={recordId} doc-type="Contract" allow-multiple="true" auto-upload="false" ondocuploaded={handleDocumentUploaded}> </c-google-drive-uploader>
```

6 Troubleshooting & Operational Matrix

Symptom / Error	Probable Cause	Resolution
401 Unauthorized	Expired OAuth token or unauthenticated Principal	Re-authenticate the External Credential Principal under Setup > Named Credentials.
403 Access Not Configured	Google Drive API is disabled in GCP	Navigate to GCP Console > API Library and ensure Google Drive API is enabled.
redirect_uri_mismatch	Callback URL in GCP does not match Salesforce	Copy the exact Callback URL from Salesforce Auth Provider and update GCP OAuth Credentials.
Apex Heap Limit Error	File size exceeds max transfer payload (~4 MB)	The service enforces a 4 MB safety check. Use Chunked Resumable Upload API for larger files.

Security Best Practice: Ensure user access to the Named Credential is governed via Permission Sets. Grant explicit access to the External Credential Principal under *Permission Sets > External Credential Principal Access*.